

TabPFN Relational Data / Knowledge-Graph Experiments

Last updated: 2026-05-15.

Workspace: </home/pasquale/workspace/alessandro>

This file is intended to be an exhaustive working log of the investigation so far. It records the scripts created or modified, the encoders used, generated datasets, commands, outputs, metrics, paper/codebase findings, and the current state of long-running FB15K-237 ranking results. Raw predictions, full CSVs, metadata JSONs, and datasets remain in the paths listed below; this note includes all aggregate results and the important per-relation/ranking summaries.

High-Level Conclusions

1. The original sequential-vs-random symmetry result was not evidence that TabPFN was sensitive to row order. It was mostly an entity-ID/ordinal encoding artifact.
2. Passing entity IDs as pandas categoricals does not make the transformer see unordered category symbols. TabPFN internally ordinal-encodes categorical values into numeric arrays before the transformer consumes them.
3. External one-hot removes ordinal ID leakage, but it does not by itself let TabPFN learn relational rules like symmetry reverse-holdout or transitivity.
4. Shared random entity embeddings give the same entity the same representation across columns, but pointwise pair encoders still fail when the target label depends on graph-level evidence outside the row.
5. Graph-derived encoders are necessary for relational inference:
 - [graph_reachability](#) solves transitivity-like same-relation reachability.
 - [relation_path](#) solves typed length-2 compositions.
 - [relation_path_k3](#) solves typed length-3 compositions such as cousin/great-grandparent.
6. CLUTRR-style noise shows the expected pattern:
 - disconnected noise barely affects path features;
 - irrelevant endpoint-attached noise hurts path features;
 - supporting/cyclic noise can help or hurt depending on whether it creates useful alternate paths.
7. On sampled FB15K-237 binary triple classification, typed path hashes substantially improve AP/AUC over pointwise encoders.
8. On exact KBC-style filtered ranking, TabPFN is far slower and much weaker than the [facebookresearch/kbc](#) ComplEx/N3 reference protocol. The best TabPFN path encoder is useful as a local candidate scorer/reranker, not as a full all-entity ranker.
9. To improve TabPFN for FB15K-237 link prediction, the next principled step is a hybrid KGE/path-rule candidate generator plus TabPFN reranker with hard negatives and reciprocal queries.

Papers / Codebases Read

TabPFN arXiv 2511.08667

User asked whether TabPFN should be invariant to training row order. The relevant distinction we used:

- Transformer/set-style row processing can be invariant/equivariant to row permutations in the model architecture.
- That does not imply invariance to arbitrary numeric encodings of category labels.
- If categories are converted to numeric ordinals, the numeric values themselves can leak structure.
- In our original sequential entity IDs, entity identifiers contained structure that TabPFN could exploit.

TabPFN Source Findings

Local source investigated under installed [tabpfn](#) package and local checkpoint repo [tabpfn_2_5](#).

Key finding:

- [TabPFNClassifier](#) calls data cleaning/preprocessing before the transformer.
- [tabpfn/preprocessing/clean.py](#) uses an ordinal encoder ([get_ordinal_encoder](#)) and converts categorical values to numeric arrays, including `astype(np.float64)`.
- Therefore pandas categoricals are not preserved as symbolic/unordered objects inside the transformer.

Consequence:

- To avoid ordinal category treatment without changing TabPFN internals, encode categories outside TabPFN:
 - external one-hot;
 - shared random embeddings;
 - graph/path relational features;
 - hashed sparse-style feature maps.

CLUTRR arXiv 1908.06177

Used for the noise taxonomy:

- irrelevant noise: distractor facts/paths attached to an endpoint;
- supporting/cyclic noise: superfluous facts involving entities already on the proof path;
- disconnected noise: facts unrelated to the query/proof path.

Adaptation here:

- CLUTRR noise is per-story.

- Our benchmark uses one global graph.
- We therefore added noise as extra background graph facts after computing clean labels, so query labels stay fixed while encoders see spurious evidence.

facebookresearch/kbc

Cloned to [external/kbc](#).

Important protocol details from the code:

- `kbc/datasets.py` evaluates both missing RHS and LHS; LHS is implemented by swapping head/tail and adding reciprocal relation offset.
- `kbc/process_datasets.py` builds filtered candidate lists from train+valid+test true triples.
- `kbc/models.py` ranks by counting candidates with score $\geq$ target score after filtering known positives and the target itself.
- README reports FB15K-237 filtered metrics for ComplEx/N3. The strong reference numbers are approximately MRR **.37**, H@1 **.27**, H@3 **.40**, H@10 **.56** for large ranks.

Files Created or Modified

Python scripts in repo root:

- `generate_minimal_symmetry_dataset.py`
 - Original/minimal symmetry reverse-holdout generator.
 - Supports sequential vs random entity IDs and demonstration-fraction splits.
- `run_minimal_symmetry_tfm_experiment.py`
 - Existing runner extended for TabPFN entity encodings.
 - Added `--tabpfn-entity-encoding` with:
 - `categorical`
 - `onehot`
 - `shared_embedding`
 - Default became `shared_embedding`.
 - Important functions:
 - `onehot_encode_entity_frame`
 - `shared_embedding_encode_entity_frame`
- `generate_relational_pattern_datasets.py`
 - Created synthetic relation-family datasets:
 - `symmetry_reverse`
 - `antisymmetry_reverse`
 - `reflexivity_identity`
 - `transitivity_chain`
 - `composition_grandparent`
 - `family_complex`
 - `family_complex_noise_irrelevant`
 - `family_complex_noise_supporting`
 - `family_complex_noise_disconnected`
 - `family_complex_noise_mixed`
 - Added query/background flags:
 - `is_query`
 - `is_background`
 - `rule`
 - Added noise flags:
 - `is_noise`
 - `noise_type`
- `run_relational_encoder_benchmark.py`
 - Created TabPFN benchmark for relational-pattern datasets.
 - Encoders:
 - `categorical`
 - `role_onehot`
 - `unordered_onehot`
 - `shared_embedding_symmetric`
 - `shared_embedding_directed`
 - `embedding_diff`
 - `graph_reachability`
 - `relation_path`
 - `relation_path_k3`
 - Added graph source separation:
 - if `is_query` exists, query rows are used for supervised train/validation/test;
 - background rows are used for graph features.
 - Outputs:

- `metrics.csv`
- `metrics_by_relation.csv`
- `metadata.json`
- `run_fb15k237_encoder_benchmark.py`
 - Created sampled FB15K-237 binary triple-classification benchmark.
 - Downloads/uses raw FB15K-237 triples from `data/fb15k237_raw`.
 - Uses one filtered relation-domain corruption per positive.
 - Encoders:
 - `categorical`
 - `hashed_role`
 - `hashed_unordered`
 - `shared_embedding_symmetric`
 - `shared_embedding_directed`
 - `embedding_diff`
 - `graph_reachability`
 - `relation_path_hash_k2`
 - `relation_path_hash_k3`
- `run_fb15k237_tabpfn_ranking.py`
 - Created exact KBC-style filtered ranking evaluator for TabPFN.
 - Scores all 14,541 candidate entities for each RHS and LHS query.
 - Builds filters from train+valid+test true triples.
 - Produces MRR, mean rank, median rank, H@1/H@3/H@10.
- `continue_fb15k237_tabpfn_ranking.py`
 - Created resumable long-running ranking continuation script.
 - Reuses already-computed ranks.
 - Prints hourly compact status.
 - Writes `hourly_status.csv` and updates `filtered_ranking_metrics.csv`.

External code:

- `external/kbc`
 - Cloned from <https://github.com/facebookresearch/kbc>.

Artifact Manifest

Important result directories:

- `symmetry_demo_fraction_datasets/`
- `tfm_results_tabpfn/`
- `relational_pattern_datasets/`
- `complex_relational_datasets/`
- `complex_relational_datasets_noisy/`
- `relational_pattern_results/`
- `fb15k237_results/`
- `data/fb15k237_raw/`
- `external/kbc/`
- `relational_pattern_results/`
- `data/fb15k237_raw/`
- `fb15k237_results/`
- `external/kbc/`

Important raw FB15K-237 files:

- `data/fb15k237_raw/train.txt`
- `data/fb15k237_raw/valid.txt`
- `data/fb15k237_raw/test.txt`

Important generated FB15K files:

- `fb15k237_results/sampled_4k_1k_2k_all_encoders/fb15k237_binary_sample.csv`
- `fb15k237_results/sampled_4k_1k_2k_all_encoders/metrics.csv`
- `fb15k237_results/sampled_4k_1k_2k_all_encoders/metrics_by_relation.csv`
- `fb15k237_results/tabpfn_filtered_ranking_path_k3_10/filtered_ranks.csv`
- `fb15k237_results/tabpfn_filtered_ranking_path_k3_10/filtered_ranking_metrics.csv`
- `fb15k237_results/tabpfn_filtered_ranking_path_k3_10/hourly_status.csv`
- `fb15k237_results/tabpfn_filtered_ranking_categorical_10/filtered_ranks.csv`
- `fb15k237_results/tabpfn_filtered_ranking_categorical_10/filtered_ranking_metrics.csv`

Encoder Definitions

Minimal Symmetry Runner Encoders

categorical

- Passes `entity_1`, `entity_2` as pandas categoricals to TabPFN.
- TabPFN still ordinal-encodes internally.

onehot

- External one-hot with role-separated columns:
 - `entity_1=<entity>`
 - `entity_2=<entity>`
- Removes ordinal leakage.
- Does not share the same column for the same entity across roles.

shared_embedding

- Assigns each entity a random vector `z_e`.
- Same entity receives the same vector independent of whether it appears as left or right.
- Symmetric pair features:

```
pair(a, b) -> [
  z_a + z_b,
  abs(z_a - z_b),
  z_a * z_b
]
```

The alternative `z_a - z_b` only was tested manually and performed poorly on symmetry because reversing the pair flips the sign.

Relational Benchmark Encoders

categorical

- Categorical `entity_1`, `entity_2`, `relation`.

role_onehot

- One-hot left role plus one-hot right role plus relation one-hot.

unordered_onehot

- One shared entity one-hot vector with both endpoints activated, plus relation one-hot.
- Symmetric by construction.
- Good for symmetry, inappropriate for directed/antisymmetric relations.

shared_embedding_symmetric

- Same as minimal shared embedding:
 - sum
 - absolute difference
 - product
- Symmetric by construction.

shared_embedding_directed

- Features:

```
[z_left, z_right, z_left - z_right, z_left * z_right]
```

embedding_diff

- Features:

```
z_left - z_right
```

graph_reachability

Built from positive train/background graph only.

Features include:

- same entity
- direct positive edge
- reverse positive edge

- direct negative edge
- reverse negative edge
- same-relation forward reachability
- same-relation reverse reachability
- inverse path distances
- direction differences
- relation one-hot

relation_path

Adds typed length-2 path counts:

```
left --r1--> middle --r2--> right
right --r1--> middle --r2--> left
```

For every relation pair (r_1 , r_2), features include:

- forward count
- reverse count
- forward exists
- reverse exists
- count direction

relation_path_k3

Extends `relation_path` with typed length-3 paths:

```
left --r1--> m1 --r2--> m2 --r3--> right
right --r1--> m1 --r2--> m2 --r3--> left
```

FB15K-237 Encoders

Dense one-hot is not practical for 14,541 entities, so hashed analogues were added.

categorical

- Raw categorical head/relation/tail.

hashed_role

- Hash bucket features for role-specific head/tail identity plus relation one-hot.

hashed_unordered

- Hash bucket features for unordered entity identity plus relation one-hot.

shared_embedding_symmetric, shared_embedding_directed, embedding_diff

- Same ideas as the synthetic benchmark.

graph_reachability

- Same-relation bounded reachability in the FB15K train graph.

relation_path_hash_k2

- Hashed typed length-2 path counts plus graph base features and relation one-hot.

relation_path_hash_k3

- Hashed typed length-2 and length-3 path counts plus graph base features and relation one-hot.

Minimal Symmetry Dataset Commands and Results

Original commands:

```
python3 generate_minimal_symmetry_dataset.py \
  --output-dir symmetry_demo_fraction_datasets/sequential_demo_0.0 \
  --seed 20260514 --num-groups 100 --group-size 10 \
  --negative-ratio 1.0 --split-mode symmetry_demonstration \
  --demo-fraction 0.0 --validation-fraction 0.2 \
  --sequential-entity-ids

python3 generate_minimal_symmetry_dataset.py \
  --output-dir symmetry_demo_fraction_datasets/random_demo_0.0 \
```

```
--seed 20260514 --num-groups 100 --group-size 10 \  
--negative-ratio 1.0 --split-mode symmetry_demonstration \  
--demo-fraction 0.0 --validation-fraction 0.2
```

Generation output for both:

```
entities=1000 rows=18000 train=9000 validation=1800 test=7200  
label_counts={'0': 9000, '1': 9000}
```

Original TabPFN result:

dataset/encoding	model	threshold	accuracy	f1	precision	recall	average_precision	roc_auc
sequential_demo_0.0 original categorical/ordinal path	tabpfn	0.500000	0.993611	0.993687	0.987452	1.000000	0.997740	0.998232
random_demo_0.0 original categorical/ordinal path	tabpfn	0.500000	0.501944	0.519561	0.502202	0.538163	0.507585	0.508145

Interpretation:

- Sequential IDs made entity identity numerically meaningful.
- Random IDs destroyed that artifact.
- This was not a row-order invariance failure.

28-Entity and 1000-Entity Symmetry Experiments

Main aggregate results recovered from `tfm_results_tabpfn/*/metrics.csv`:

path	model	threshold	accuracy	f1
<code>tfm_results_tabpfn/results_sequential_demo_0.0/metrics.csv</code>	tabpfn	0.500000	0.993611	0.993687
<code>tfm_results_tabpfn/results_random_demo_0.0/metrics.csv</code>	tabpfn	0.500000	0.501944	0.519561
<code>tfm_results_tabpfn/results_sequential_28entities_demo_0.0/metrics.csv</code>	tabpfn	0.500000	0.895522	0.898551
<code>tfm_results_tabpfn/results_random_28entities_demo_0.0/metrics.csv</code>	tabpfn	0.500000	0.537313	0.523077
<code>tfm_results_tabpfn/results_sequential_28entities_demo_0.0_categorical/metrics.csv</code>	tabpfn	0.500000	0.895522	0.898551
<code>tfm_results_tabpfn/results_random_28entities_demo_0.0_categorical/metrics.csv</code>	tabpfn	0.500000	0.537313	0.523077
<code>tfm_results_tabpfn/results_sequential_28entities_demo_0.0_onehot/metrics.csv</code>	tabpfn	0.500000	0.432836	0.441176
<code>tfm_results_tabpfn/results_random_28entities_demo_0.0_onehot/metrics.csv</code>	tabpfn	0.500000	0.447761	0.478873
<code>tfm_results_tabpfn/results_sequential_28entities_demo_0.0_shared_embedding/metrics.csv</code>	tabpfn	0.500000	0.955224	0.956522
<code>tfm_results_tabpfn/results_random_28entities_demo_0.0_shared_embedding/metrics.csv</code>	tabpfn	0.500000	0.985075	0.985507
<code>tfm_results_tabpfn/results_sequential_demo_0.0_onehot/metrics.csv</code>	tabpfn	0.500000	0.495417	0.634397
<code>tfm_results_tabpfn/results_random_demo_0.0_onehot/metrics.csv</code>	tabpfn	0.500000	0.500556	0.665177
<code>tfm_results_tabpfn/results_sequential_demo_0.0_shared_embedding/metrics.csv</code>	tabpfn	0.500000	0.537639	0.617136
<code>tfm_results_tabpfn/results_random_demo_0.0_shared_embedding/metrics.csv</code>	tabpfn	0.500000	0.543611	0.548502

Manual `z_a - z_b` only test:

dataset	result summary
sequential 28	accuracy <code>.313</code> , AUC <code>.242</code>
random 28	accuracy <code>.403</code> , AUC <code>.378</code>
sequential 1000	accuracy <code>.464</code> , AUC <code>.452</code>
random 1000	accuracy <code>.482</code> , AUC <code>.464</code>

Interpretation:

- `a - b` flips sign under reversal and is therefore a bad encoder for symmetric relations.
- Shared symmetric embeddings can work on tiny 28-entity datasets, but scale poorly to 1000 entities because the model still lacks graph-level relational evidence.

Synthetic Relational Pattern Datasets

Generated by:

```
python3 generate_relational_pattern_datasets.py \
  --output-dir relational_pattern_datasets \
  --seed 20260514 \
  --num-entities 64 \
  --num-groups 8 \
  --group-size 8 \
  --validation-fraction 0.2
```

Dataset summaries:

dataset	rows	split counts	label counts / relation notes
symmetry_reverse	896	train 448, validation 90, test 358	relation same_group
antisymmetry_reverse	896	train 448, validation 90, test 358	relation less_than
reflexivity_identity	128	train 77, validation 26, test 25	relation identity
transitivity_chain	510	train 126, validation 76, test 308	relation ancestor
composition_grandparent	208	train 160, validation 9, test 39	relations parent , grandparent

Initial All-Encoder Synthetic Relational Benchmark

From [relational_pattern_results/tabPFN_all_encoders_tuned/metrics.csv](#).

dataset	encoder	train_rows	validation_rows	test_rows	n_features	threshold	accuracy	f1
antisymmetry_reverse	categorical	448	90	358	3	0.550000	0.689944	0.683761
antisymmetry_reverse	role_onehot	448	90	358	129	0.730000	0.796089	0.730627
antisymmetry_reverse	unordered_onehot	448	90	358	65	0.000000	0.463687	0.633588
antisymmetry_reverse	shared_embedding_symmetric	448	90	358	193	0.000000	0.463687	0.633588
antisymmetry_reverse	shared_embedding_directed	448	90	358	257	0.700000	0.796089	0.750853
antisymmetry_reverse	embedding_diff	448	90	358	65	0.540000	0.877095	0.865854
composition_grandparent	categorical	160	9	39	3	0.000000	0.461538	0.631579
composition_grandparent	role_onehot	160	9	39	130	0.347168	0.717949	0.717949
composition_grandparent	unordered_onehot	160	9	39	66	0.000000	0.461538	0.631579
composition_grandparent	shared_embedding_symmetric	160	9	39	194	0.000000	0.461538	0.631579
composition_grandparent	shared_embedding_directed	160	9	39	258	0.337729	0.820513	0.787879
composition_grandparent	embedding_diff	160	9	39	66	0.345543	0.794872	0.714286
reflexivity_identity	categorical	77	26	25	3	0.002960	0.480000	0.580645
reflexivity_identity	role_onehot	77	26	25	129	0.000000	0.360000	0.529412
reflexivity_identity	unordered_onehot	77	26	25	65	0.000000	0.360000	0.529412
reflexivity_identity	shared_embedding_symmetric	77	26	25	193	0.010000	1.000000	1.000000
reflexivity_identity	shared_embedding_directed	77	26	25	257	0.010000	0.960000	0.947368
reflexivity_identity	embedding_diff	77	26	25	65	0.010000	0.960000	0.947368
symmetry_reverse	categorical	448	90	358	3	0.000000	0.491620	0.659176
symmetry_reverse	role_onehot	448	90	358	129	0.464486	0.790503	0.789916
symmetry_reverse	unordered_onehot	448	90	358	65	0.260000	0.994413	0.994350
symmetry_reverse	shared_embedding_symmetric	448	90	358	193	0.498535	0.882682	0.884615
symmetry_reverse	shared_embedding_directed	448	90	358	257	0.460000	0.516760	0.658777
symmetry_reverse	embedding_diff	448	90	358	65	0.470000	0.502793	0.661597
transitivity_chain	categorical	126	76	308	3	0.471191	0.493506	0.654867
transitivity_chain	role_onehot	126	76	308	129	0.499146	0.467532	0.594059
transitivity_chain	unordered_onehot	126	76	308	65	0.000000	0.500000	0.666667
transitivity_chain	shared_embedding_symmetric	126	76	308	193	0.000000	0.500000	0.666667
transitivity_chain	shared_embedding_directed	126	76	308	257	0.498535	0.516234	0.642686
transitivity_chain	embedding_diff	126	76	308	65	0.000000	0.500000	0.666667

Interpretation:

- Symmetry: `unordered_onehot` and `shared_embedding_symmetric` work.
- Antisymmetry: directed/difference features work; symmetric features fail.
- Reflexivity: embedding equality/difference features work.
- Composition: pointwise features can rank scores somewhat but not principled.
- Transitivity: all pointwise encoders are near chance; need graph features.

Graph Reachability

From `relational_pattern_results/graph_reachability_all/metrics.csv`.

dataset	encoder	n_features	threshold	accuracy	f1	average_precision	roc_auc
antisymmetry_reverse	graph_reachability	11	0.112413	1.000000	1.000000	1.000000	1.000000
composition_grandparent	graph_reachability	12	0.180000	0.564103	0.190476	0.602449	0.633598
reflexivity_identity	graph_reachability	11	0.280899	0.320000	0.484848	0.349368	0.451389
symmetry_reverse	graph_reachability	11	0.120000	1.000000	1.000000	1.000000	1.000000
transitivity_chain	graph_reachability	11	0.290000	1.000000	1.000000	1.000000	1.000000

After adding `graph_same_entity`, graph reachability also solved reflexivity:

dataset	graph_reachability f1	relation_path f1
antisymmetry_reverse	1.000000	1.000000
composition_grandparent	0.190476	1.000000
reflexivity_identity	1.000000	1.000000
symmetry_reverse	1.000000	1.000000
transitivity_chain	1.000000	1.000000

Relation Path With Equality

From `relational_pattern_results/relation_path_all_with_equality/metrics.csv`.

dataset	encoder	n_features	threshold	accuracy	f1	average_precision	roc_auc
antisymmetry_reverse	relation_path	17	0.160000	1.000000	1.000000	1.000000	1.000000
composition_grandparent	relation_path	33	0.080000	1.000000	1.000000	1.000000	1.000000
reflexivity_identity	relation_path	17	0.120000	1.000000	1.000000	1.000000	1.000000
symmetry_reverse	relation_path	17	0.090000	1.000000	1.000000	1.000000	1.000000
transitivity_chain	relation_path	17	0.290000	1.000000	1.000000	1.000000	1.000000

Key point:

- `graph_reachability` solved transitivity.
- `relation_path` solved typed composition (`grandparent = parent, parent`).
- `graph_same_entity` was necessary for reflexivity.

Complex Family Dataset

Created `family_complex` in `generate_relational_pattern_datasets.py`.

Background relations:

- `parent`
- `child`
- `sibling`
- `spouse`

Query relations:

- `grandparent = parent, parent`
- `great_grandparent = parent, parent, parent`
- `aunt_uncle = sibling, parent`
- `nibling = child, sibling`
- `cousin = child, sibling, parent`
- `parent_in_law = parent, spouse`
- `sibling_in_law = sibling, spouse OR spouse, sibling`
- `ancestor = parent, parent OR parent, parent, parent`

Clean dataset summary:

item	count
total rows	5152
background rows	1696
query rows	3456
train query rows	1728
validation query rows	692
test query rows	1036
test positives	518
test negatives	518

Family Complex All Encoders

From `relational_pattern_results/family_complex_all_encoders/metrics.csv`.

dataset	encoder	graph_rows	n_features	threshold	accuracy	f1	precision	recall	average_prec
family_complex	categorical	1696	3	0.420000	0.525097	0.668910	0.513430	0.959459	0.53
family_complex	role_onehot	1696	812	0.434570	0.532819	0.661538	0.518640	0.913127	0.57
family_complex	unordered_onehot	1696	412	0.345543	0.597490	0.695398	0.559342	0.918919	0.75
family_complex	shared_embedding_symmetric	1696	204	0.429827	0.517375	0.657534	0.509554	0.926641	0.58
family_complex	shared_embedding_directed	1696	268	0.426270	0.623552	0.669492	0.596677	0.762548	0.65
family_complex	embedding_diff	1696	76	0.452637	0.521236	0.664411	0.511458	0.947876	0.52
family_complex	graph_reachability	1696	23	0.000000	0.500000	0.666667	0.500000	1.000000	0.50
family_complex	relation_path	1696	103	0.330000	0.873552	0.883140	0.820896	0.955598	0.95
family_complex	relation_path_k3	1696	423	0.750000	1.000000	1.000000	1.000000	1.000000	1.00

Per-relation F1 for path encoders:

relation	relation_path	relation_path_k3
ancestor	0.809917	1.000000
aunt_uncle	1.000000	1.000000
cousin	0.672897	1.000000
grandparent	1.000000	1.000000
great_grandparent	0.672566	1.000000
nibling	1.000000	1.000000
parent_in_law	1.000000	1.000000
sibling_in_law	0.993103	1.000000

Interpretation:

- `relation_path` solves length-2 relations.
- It misses length-3 rules like `cousin` and `great_grandparent`.
- `relation_path_k3` solves all generated clean rules.

CLUTRR-Style Noisy Family Datasets

Generated variants:

- `family_complex`
- `family_complex_noise_irrelevant`
- `family_complex_noise_supporting`
- `family_complex_noise_disconnected`
- `family_complex_noise_mixed`

Noise counts:

dataset	total rows	background rows	query rows	noise rows	noise types
family_complex	5152	1696	3456	0	clean only

dataset	total rows	background rows	query rows	noise rows	noise types
family_complex_noise_irrelevant	7552	4096	3456	2400	irrelevant
family_complex_noise_supporting	7552	4096	3456	2400	supporting
family_complex_noise_disconnected	7552	4096	3456	2400	disconnected
family_complex_noise_mixed	12352	8896	3456	7200	irrelevant + supporting + disconnected

All noisy variants have the exact same query rows as clean.

Noisy Aggregate Results

From relational_pattern_results/family_complex_noisy_all_encoders/metrics.csv.

dataset	encoder	graph_rows	n_features	threshold	accuracy	f1	precision	re
family_complex	categorical	1696	3	0.473633	0.509653	0.658602	0.505155	0.945
family_complex	role_onehot	1696	812	0.434082	0.520270	0.665320	0.510858	0.953
family_complex	unordered_onehot	1696	412	0.443848	0.666988	0.705380	0.632466	0.797
family_complex	shared_embedding_symmetric	1696	204	0.000000	0.500000	0.666667	0.500000	1.000
family_complex	shared_embedding_directed	1696	268	0.245117	0.540541	0.680108	0.521649	0.976
family_complex	embedding_diff	1696	76	0.441270	0.510618	0.667541	0.505462	0.982
family_complex	graph_reachability	1696	23	0.000000	0.500000	0.666667	0.500000	1.000
family_complex	relation_path	1696	103	0.220000	0.867761	0.878222	0.813839	0.953
family_complex	relation_path_k3	1696	423	0.750000	0.999035	0.999036	0.998073	1.000
family_complex_noise_irrelevant	categorical	4096	3	0.473761	0.511583	0.659489	0.506198	0.945
family_complex_noise_irrelevant	role_onehot	4096	812	0.418457	0.509653	0.664021	0.505030	0.969
family_complex_noise_irrelevant	unordered_onehot	4096	412	0.443848	0.668919	0.706587	0.634409	0.797
family_complex_noise_irrelevant	shared_embedding_symmetric	4096	204	0.000000	0.500000	0.666667	0.500000	1.000
family_complex_noise_irrelevant	shared_embedding_directed	4096	268	0.228421	0.530888	0.676862	0.516227	0.982
family_complex_noise_irrelevant	embedding_diff	4096	76	0.441406	0.512548	0.668418	0.506468	0.982
family_complex_noise_irrelevant	graph_reachability	4096	23	0.000000	0.500000	0.666667	0.500000	1.000
family_complex_noise_irrelevant	relation_path	4096	103	0.487305	0.853282	0.862816	0.810169	0.922
family_complex_noise_irrelevant	relation_path_k3	4096	423	0.580000	0.979730	0.979788	0.976967	0.982
family_complex_noise_supporting	categorical	4096	3	0.473761	0.511583	0.659489	0.506198	0.945
family_complex_noise_supporting	role_onehot	4096	812	0.418457	0.509653	0.664021	0.505030	0.969
family_complex_noise_supporting	unordered_onehot	4096	412	0.443848	0.668919	0.706587	0.634409	0.797
family_complex_noise_supporting	shared_embedding_symmetric	4096	204	0.000000	0.500000	0.666667	0.500000	1.000
family_complex_noise_supporting	shared_embedding_directed	4096	268	0.228421	0.530888	0.676862	0.516227	0.982
family_complex_noise_supporting	embedding_diff	4096	76	0.441406	0.512548	0.668418	0.506468	0.982
family_complex_noise_supporting	graph_reachability	4096	23	0.000000	0.500000	0.666667	0.500000	1.000
family_complex_noise_supporting	relation_path	4096	103	0.400000	0.946911	0.947569	0.935970	0.959
family_complex_noise_supporting	relation_path_k3	4096	423	0.480000	0.969112	0.969925	0.945055	0.996
family_complex_noise_disconnected	categorical	4096	3	0.473761	0.511583	0.659489	0.506198	0.945
family_complex_noise_disconnected	role_onehot	4096	1068	0.439590	0.521236	0.663043	0.511530	0.942
family_complex_noise_disconnected	unordered_onehot	4096	540	0.390000	0.521236	0.665768	0.511387	0.953
family_complex_noise_disconnected	shared_embedding_symmetric	4096	204	0.302905	0.500000	0.664942	0.500000	0.992
family_complex_noise_disconnected	shared_embedding_directed	4096	268	0.284354	0.539575	0.672165	0.521878	0.944
family_complex_noise_disconnected	embedding_diff	4096	76	0.431152	0.521236	0.653631	0.512035	0.903
family_complex_noise_disconnected	graph_reachability	4096	23	0.000000	0.500000	0.666667	0.500000	1.000
family_complex_noise_disconnected	relation_path	4096	103	0.220000	0.867761	0.878222	0.813839	0.953
family_complex_noise_disconnected	relation_path_k3	4096	423	0.720000	0.999035	0.999036	0.998073	1.000

dataset	encoder	graph_rows	n_features	threshold	accuracy	f1	precision	re
family_complex_noise_mixed	categorical	8896	3	0.473145	0.509653	0.659060	0.505144	0.947
family_complex_noise_mixed	role_onehot	8896	1068	0.436661	0.519305	0.661224	0.510504	0.938
family_complex_noise_mixed	unordered_onehot	8896	540	0.390000	0.522201	0.666217	0.511917	0.953
family_complex_noise_mixed	shared_embedding_symmetric	8896	204	0.310000	0.499035	0.664078	0.499513	0.990
family_complex_noise_mixed	shared_embedding_directed	8896	268	0.286133	0.540541	0.672176	0.522484	0.942
family_complex_noise_mixed	embedding_diff	8896	76	0.423828	0.521236	0.658872	0.511752	0.924
family_complex_noise_mixed	graph_reachability	8896	23	0.000000	0.500000	0.666667	0.500000	1.000
family_complex_noise_mixed	relation_path	8896	103	0.250000	0.915058	0.918969	0.878521	0.963
family_complex_noise_mixed	relation_path_k3	8896	423	0.360000	0.948842	0.950421	0.921960	0.980

Path-encoder per-relation F1 under noise:

dataset	relation	relation_path	relation_path_k3
family_complex	ancestor	0.800000	1.000000
family_complex	aunt_uncle	0.979592	0.993103
family_complex	cousin	0.669767	1.000000
family_complex	grandparent	1.000000	1.000000
family_complex	great_grandparent	0.666667	1.000000
family_complex	nibbling	1.000000	1.000000
family_complex	parent_in_law	0.989691	1.000000
family_complex	sibling_in_law	1.000000	1.000000
family_complex_noise_irrelevant	ancestor	0.800000	0.945205
family_complex_noise_irrelevant	aunt_uncle	0.986301	0.993103
family_complex_noise_irrelevant	cousin	0.600000	0.923077
family_complex_noise_irrelevant	grandparent	0.986301	1.000000
family_complex_noise_irrelevant	great_grandparent	0.609524	1.000000
family_complex_noise_irrelevant	nibbling	1.000000	1.000000
family_complex_noise_irrelevant	parent_in_law	0.989691	1.000000
family_complex_noise_irrelevant	sibling_in_law	0.993103	0.993103
family_complex_noise_supporting	ancestor	0.932432	0.986111
family_complex_noise_supporting	aunt_uncle	0.979592	0.960000
family_complex_noise_supporting	cousin	0.805755	0.953642
family_complex_noise_supporting	grandparent	0.993103	0.986301
family_complex_noise_supporting	great_grandparent	0.935065	0.974359
family_complex_noise_supporting	nibbling	0.972973	0.953020
family_complex_noise_supporting	parent_in_law	0.989691	0.979592
family_complex_noise_supporting	sibling_in_law	0.972973	0.972973
family_complex_noise_disconnected	ancestor	0.800000	1.000000
family_complex_noise_disconnected	aunt_uncle	0.979592	0.993103
family_complex_noise_disconnected	cousin	0.669767	1.000000
family_complex_noise_disconnected	grandparent	1.000000	1.000000
family_complex_noise_disconnected	great_grandparent	0.666667	1.000000
family_complex_noise_disconnected	nibbling	1.000000	1.000000
family_complex_noise_disconnected	parent_in_law	0.989691	1.000000
family_complex_noise_disconnected	sibling_in_law	1.000000	1.000000
family_complex_noise_mixed	ancestor	0.910345	0.972603
family_complex_noise_mixed	aunt_uncle	0.953642	0.940397

dataset	relation	relation_path	relation_path_k3
family_complex_noise_mixed	cousin	0.753086	0.897959
family_complex_noise_mixed	grandparent	0.986301	0.993103
family_complex_noise_mixed	great_grandparent	0.867470	0.936709
family_complex_noise_mixed	nibling	0.966443	0.953642
family_complex_noise_mixed	parent_in_law	0.941176	0.960000
family_complex_noise_mixed	sibling_in_law	0.972973	0.946667

Interpretation:

- **relation_path_k3** remains best overall but is not perfectly robust to endpoint-attached/mixed noise.
- Disconnected noise leaves path encoders essentially unchanged.
- Supporting noise can improve **relation_path** because it creates alternate short paths.
- Pointwise encoders remain weak.

FB15K-237 Binary Triple Classification

Raw FB15K-237 downloaded to:

```
data/fb15k237_raw/train.txt
data/fb15k237_raw/valid.txt
data/fb15k237_raw/test.txt
```

Raw split sizes:

split	triples
train	272115
valid	17535
test	20466

Metadata from `fb15k237_results/sampled_4k_1k_2k_all_encoders/metadata.json`:

item	value
entities	14541
relations	237
sampled train positives	4000
sampled validation positives	1000
sampled test positives	2000
negative ratio	1.0
negative sampling	filtered relation-domain head/tail corruption using train relation domains
background graph	FB15K-237 train triples excluding sampled train query positives

Command:

```
python3 run_fb15k237_encoder_benchmark.py \
  --raw-dir data/fb15k237_raw \
  --output-dir fb15k237_results/sampled_4k_1k_2k_all_encoders \
  --encoders all \
  --train-positives 4000 \
  --valid-positives 1000 \
  --test-positives 2000 \
  --n-estimators 2 \
  --device cuda \
  --tabpfn-model-path tabpfn_2_5/tabpfn-v2.5-classifier-v2.5_default.ckpt \
  --threshold-mode tune_validation_f1
```

Results:

encoder	train_rows	validation_rows	test_rows	n_features	threshold	feature_time_s	fit_time_s	predict_time_s
categorical	8000	2000	4000	3	0.454368	0.009394	0.331690	1.424206

encoder	train_rows	validation_rows	test_rows	n_features	threshold	feature_time_s	fit_time_s	predict_time_s
hashed_role	8000	2000	4000	493	0.410000	0.044710	0.992214	40.228828
hashed_unordered	8000	2000	4000	493	0.440000	0.043266	1.074797	40.119961
shared_embedding_symmetric	8000	2000	4000	429	0.392918	0.024038	0.909367	37.807955
shared_embedding_directed	8000	2000	4000	493	0.427595	0.021269	1.171698	40.269701
embedding_diff	8000	2000	4000	301	0.470574	0.018286	0.981582	26.613895
graph_reachability	8000	2000	4000	246	0.260000	0.168521	0.588233	26.662783
relation_path_hash_k2	8000	2000	4000	502	0.280273	2.004470	1.171458	40.571551
relation_path_hash_k3	8000	2000	4000	758	0.288086	13.838815	1.131974	43.964016

Interpretation:

- Pointwise encoders are weak.
- Same-relation reachability is weak on FB15K-237.
- Typed path hashes provide a large AP/AUC gain.
- `relation_path_hash_k3` is best among tested binary classifiers.

FB15K-237 Exact KBC-Style Filtered Ranking

Ranking script:

```
python3 run_fb15k237_tabpfn_ranking.py ...
```

Protocol:

- Score all 14,541 entities for RHS prediction (`h, r, ?`) and LHS prediction (`?, r, t`).
- Filter known true triples from train+valid+test.
- Rank convention follows [facebookresearch/kbc](#): rank is `1 + count(scores >= target_score)` after filtering.
- Metrics:
 - MRR
 - mean rank
 - median rank
 - H@1
 - H@3
 - H@10

Full exact ranking cost estimate:

```
20466 test triples * 2 sides * 14541 entities = about 595M candidate triples
```

Measured `relation_path_hash_k3` cost:

```
about 90 seconds per test triple
```

So full exact evaluation would take on the order of weeks on this machine.

10-Triple Exact Ranking Subset

categorical:

encoder	eval_triples	eval_queries	both_mrr	both_hits@1	both_hits@3	both_hits@10	rhs_mrr	lhs_mrr	ranking_time_s
categorical	10	20	0.000389	0.000000	0.000000	0.000000	0.000529	0.000250	17.715640

`relation_path_hash_k3` initial 10-triple run:

encoder	eval_triples	eval_queries	both_mrr	both_hits@1	both_hits@3	both_hits@10	rhs_mrr	lhs_mrr	ranking_time_s
relation_path_hash_k3	10	20	0.060341	0.000000	0.100000	0.150000	0.052715	0.067967	972.32045

Ranks for those 10 path-k3 triples:

query_index	side	relation	rank	target_score	num_fi
1	rhs	<code>/people/person/religion</code>	900	0.533203	

query_index	side	relation	rank	target_score	num_fi
1	lhs	/people/person/religion	3351	0.533203	
2	rhs	/education/educational_institution_campus/educational_institution	3	0.702320	
2	lhs	/education/educational_institution_campus/educational_institution	14	0.702320	
3	rhs	/people/person/profession	10	0.894859	
3	lhs	/people/person/profession	63	0.894859	
4	rhs	/olympics/olympic_games/participating_countries	94	0.753722	
4	lhs	/olympics/olympic_games/participating_countries	47	0.753722	
5	rhs	/location/location/contains	231	0.886989	
5	lhs	/location/location/contains	2	0.886989	
6	rhs	/film/film/release_date_s./film/film_regional_release_date/film_release_region	1629	0.351074	
6	lhs	/film/film/release_date_s./film/film_regional_release_date/film_release_region	2406	0.351074	
7	rhs	/award/award_nominee/award_nominations./award/award_nomination/nominated_for	14	0.734554	
7	lhs	/award/award_nominee/award_nominations./award/award_nomination/nominated_for	15	0.734554	
8	rhs	/film/film/country	353	0.320312	
8	lhs	/film/film/country	4922	0.320312	
9	rhs	/film/film/language	358	0.516964	
9	lhs	/film/film/language	291	0.516964	
10	rhs	/business/business_operation/industry	14402	0.319980	
10	lhs	/business/business_operation/industry	14301	0.319980	

Continued Long Ranking Run

User asked to run for 6 hours with hourly updates; it was stopped at user request after the 4-hour report plus some additional completed queries.

Script:

```
python3 continue_fb15k237_tabpfn_ranking.py \
  --raw-dir data/fb15k237_raw \
  --output-dir fb15k237_results/tabpfn_filtered_ranking_path_k3_10 \
  --encoder_relation_path_hash_k3 \
  --train-positives 4000 \
  --valid-positives 1000 \
  --test-positives 2000 \
  --max-seconds 21600 \
  --report-interval-seconds 3600 \
  --n-estimators 2 \
  --device cuda \
  --tabpfn-model-path tabpfn_2_5/tabpfn-v2.5-classifier-v2.5_default.ckpt
```

Hourly status from `hourly_status.csv`:

phase	elapsed_h	eval_triples	eval_queries	MRR	H@1	H@3	H@10	rhs_mrr	lhs_mrr
start	0.00	10	20	0.060341	0.000000	0.100000	0.150000	0.052715	0.067967
hourly	1.02	49	98	0.072950	0.030612	0.061224	0.163265	0.081947	0.063954
hourly	2.02	88	176	0.056329	0.017045	0.051136	0.142045	0.058582	0.054075
hourly	3.03	127	254	0.052848	0.011811	0.051181	0.149606	0.056346	0.049350
hourly	4.00	164	328	0.048727	0.009146	0.048780	0.143293	0.052641	0.044813

After the user stopped the run, `filtered_ranks.csv` already contained more completed ranks than the last hourly metrics snapshot:

state	eval_triples	eval_queries	MRR	H@1	H@3	H@10	mean_rank	median_rank
current ranks file, both sides	175	350	0.046592	0.008571	0.045714	0.140000	2224.634286	534.000000
current ranks file, rhs only	175	175	0.050960	0.011429	0.045714	0.165714	1928.485714	319.000000
current ranks file, lhs only	175	175	0.042225	0.005714	0.045714	0.114286	2520.782857	895.000000

Process stop:

- Long process was found as PID **3816458**.
- It was killed cleanly.

FB15K-237 Interpretation and Proposed Improvements

Why TabPFN is weak at KBC ranking here:

- TabPFN is a row-wise tabular classifier, not a decomposable scoring model over all entities.
- Exact KBC ranking requires scoring every candidate entity for every query side.
- TabPFN feature construction plus prediction over all candidates is slow.
- Binary classification with random/relation-domain corruptions does not align well with all-entity rank ordering.
- The raw entity encoders do not learn reusable relation semantics.

Recommended path to improve link prediction:

1. Use reciprocal queries:

```
(h, r, ?)      tail prediction
(t, r^-1, ?)   head prediction
```

2. Use hard negatives:

- same relation domain/range;
- high path-score candidates;
- high-degree candidates;
- top candidates from ComplEx/DistMult/RotatE;
- type-compatible false entities.

3. Mine relation-specific rules rather than hashing all paths:

```
r(x,y) <- p1(x,z), p2(z,y)
r(x,y) <- p1(x,z1), p2(z1,z2), p3(z2,y)
```

Keep rule support/confidence features.

4. Add KG statistics:

- head/tail type/domain/range compatibility;
- relation functionality and inverse functionality;
- entity in/out degree;
- **(h, r, ?)** and **(?, r, t)** fanout counts;
- relation-specific head/tail frequency.

5. Use KGE scores as TabPFN features:

- ComplEx score;
- DistMult score;
- RotatE score;
- rank percentile;
- score margins;
- ensemble of KGE/path-rule scores.

6. Use TabPFN as a top-K reranker:

- KGE/path model proposes top 200-1000 candidates.
- TabPFN reranks candidates with rich tabular features.
- This avoids full all-entity TabPFN scoring.

7. Use more training data and a large-sample checkpoint:

- Current binary FB15K run used 4000 positives / 8000 total train rows.
- Larger relation-balanced samples should help if runtime permits.

Access Control Change

User asked to give Linux user **alessandro** access to the current folder.

Commands run:

```
setfacl -m u:alessandro:--x /home/pasquale
setfacl -m u:alessandro:--x /home/pasquale/workspace
```

```
setfacl -R -m u:alessandro:rwX /home/pasquale/workspace/alessandro
setfacl -R -d -m u:alessandro:rwX /home/pasquale/workspace/alessandro
```

Result:

- `alessandro` has traverse access through `/home/pasquale` and `/home/pasquale/workspace`.
- `alessandro` has recursive `rwX` access to `/home/pasquale/workspace/alessandro`.
- Default ACLs grant access for newly created files/directories under the workspace folder.

Reproducibility Notes

TabPFN checkpoint used:

```
tabpfn_2_5/tabpfn-v2.5-classifier-v2.5_default.ckpt
```

Typical TabPFN options used:

```
--device cuda
--seed 20260514
--threshold-mode tune_validation_f1
--tabpfn-fit-mode fit_preprocessors
--tabpfn-memory-saving-mode auto
--tabpfn-inference-precision auto
```

For FB15K binary classification:

```
--train-positives 4000
--valid-positives 1000
--test-positives 2000
--negative-ratio 1.0
--n-estimators 2
--embedding-dim 64
--hash-dim 256
--path-hash-dim 128
--max-bfs-depth 3
--max-paths-per-row 8000
```

For exact FB15K ranking:

```
all 14541 entities are scored for each RHS query
all 14541 entities are scored for each LHS query
filters are built from train+valid+test true triples
```

Complete Generated Result-File Index

The aggregate metrics, commands, encoder definitions, and conclusions are recorded in this note. The following generated files are the complete saved result/dataset artifact index at the time this note was written; row-level predictions, raw sampled triples, and ranking rows are preserved verbatim in these files.

```
./fb15k237_results/ranking_smoke_categorical/eval_triples.csv
./fb15k237_results/ranking_smoke_categorical/filtered_ranking_metrics.csv
./fb15k237_results/ranking_smoke_categorical/filtered_ranks.csv
./fb15k237_results/ranking_smoke_categorical/metadata.json
./fb15k237_results/ranking_smoke_path_k3/eval_triples.csv
./fb15k237_results/ranking_smoke_path_k3/filtered_ranking_metrics.csv
./fb15k237_results/ranking_smoke_path_k3/filtered_ranks.csv
./fb15k237_results/ranking_smoke_path_k3/metadata.json
./fb15k237_results/sampled_4k_1k_2k_all_encoders/fb15k237_binary_sample.csv
./fb15k237_results/sampled_4k_1k_2k_all_encoders/metadata.json
./fb15k237_results/sampled_4k_1k_2k_all_encoders/metrics.csv
./fb15k237_results/sampled_4k_1k_2k_all_encoders/metrics_by_relation.csv
./fb15k237_results/smoke/fb15k237_binary_sample.csv
./fb15k237_results/smoke/metadata.json
./fb15k237_results/smoke/metrics.csv
./fb15k237_results/smoke/metrics_by_relation.csv
./fb15k237_results/tabpfn_filtered_ranking_categorical_10/eval_triples.csv
./fb15k237_results/tabpfn_filtered_ranking_categorical_10/filtered_ranking_metrics.csv
./fb15k237_results/tabpfn_filtered_ranking_categorical_10/filtered_ranks.csv
./fb15k237_results/tabpfn_filtered_ranking_categorical_10/metadata.json
./fb15k237_results/tabpfn_filtered_ranking_path_k3_10/eval_triples.csv
```


./fb15k237_results/tabpfn_filtered_ranking_path_k3_10/filtered_ranking_metrics.csv
./fb15k237_results/tabpfn_filtered_ranking_path_k3_10/filtered_ranks.csv
./fb15k237_results/tabpfn_filtered_ranking_path_k3_10/hourly_status.csv
./fb15k237_results/tabpfn_filtered_ranking_path_k3_10/metadata.json
./relational_pattern_datasets/antisymmetry_reverse/metadata.json
./relational_pattern_datasets/antisymmetry_reverse/relational_atoms.csv
./relational_pattern_datasets/composition_grandparent/metadata.json
./relational_pattern_datasets/composition_grandparent/relational_atoms.csv
./relational_pattern_datasets/metadata.json
./relational_pattern_datasets/reflexivity_identity/metadata.json
./relational_pattern_datasets/reflexivity_identity/relational_atoms.csv
./relational_pattern_datasets/symmetry_reverse/metadata.json
./relational_pattern_datasets/symmetry_reverse/relational_atoms.csv
./relational_pattern_datasets/transitivity_chain/metadata.json
./relational_pattern_datasets/transitivity_chain/relational_atoms.csv
./relational_pattern_results/composition_relation_path/metadata.json
./relational_pattern_results/composition_relation_path/metrics.csv
./relational_pattern_results/family_complex_all_encoders/metadata.json
./relational_pattern_results/family_complex_all_encoders/metrics.csv
./relational_pattern_results/family_complex_all_encoders/metrics_by_relation.csv
./relational_pattern_results/family_complex_noisy_all_encoders/metadata.json
./relational_pattern_results/family_complex_noisy_all_encoders/metrics.csv
./relational_pattern_results/family_complex_noisy_all_encoders/metrics_by_relation.csv
./relational_pattern_results/graph_reachability_all/metadata.json
./relational_pattern_results/graph_reachability_all/metrics.csv
./relational_pattern_results/graph_vs_relation_path_with_equality/metadata.json
./relational_pattern_results/graph_vs_relation_path_with_equality/metrics.csv
./relational_pattern_results/relation_path_all/metadata.json
./relational_pattern_results/relation_path_all/metrics.csv
./relational_pattern_results/relation_path_all_with_equality/metadata.json
./relational_pattern_results/relation_path_all_with_equality/metrics.csv
./relational_pattern_results/tabpfn_all_encoders/metadata.json
./relational_pattern_results/tabpfn_all_encoders/metrics.csv
./relational_pattern_results/tabpfn_all_encoders_tuned/metadata.json
./relational_pattern_results/tabpfn_all_encoders_tuned/metrics.csv
./relational_pattern_results/transitivity_graph_reachability/metadata.json
./relational_pattern_results/transitivity_graph_reachability/metrics.csv
./symmetry_demo_fraction_datasets/random_28entities_demo_0.0/entities.csv
./symmetry_demo_fraction_datasets/random_28entities_demo_0.0/metadata.json
./symmetry_demo_fraction_datasets/random_28entities_demo_0.0/pair_assignments.csv
./symmetry_demo_fraction_datasets/random_28entities_demo_0.0/symmetry_friendship_atoms.csv
./symmetry_demo_fraction_datasets/random_28entities_demo_0.0/validation_report.md
./symmetry_demo_fraction_datasets/random_demo_0.0/entities.csv
./symmetry_demo_fraction_datasets/random_demo_0.0/metadata.json
./symmetry_demo_fraction_datasets/random_demo_0.0/pair_assignments.csv
./symmetry_demo_fraction_datasets/random_demo_0.0/symmetry_friendship_atoms.csv
./symmetry_demo_fraction_datasets/random_demo_0.0/validation_report.md
./symmetry_demo_fraction_datasets/sequential_28entities_demo_0.0/entities.csv
./symmetry_demo_fraction_datasets/sequential_28entities_demo_0.0/metadata.json
./symmetry_demo_fraction_datasets/sequential_28entities_demo_0.0/pair_assignments.csv
./symmetry_demo_fraction_datasets/sequential_28entities_demo_0.0/symmetry_friendship_atoms.csv
./symmetry_demo_fraction_datasets/sequential_28entities_demo_0.0/validation_report.md
./symmetry_demo_fraction_datasets/sequential_demo_0.0/entities.csv
./symmetry_demo_fraction_datasets/sequential_demo_0.0/metadata.json
./symmetry_demo_fraction_datasets/sequential_demo_0.0/pair_assignments.csv
./symmetry_demo_fraction_datasets/sequential_demo_0.0/symmetry_friendship_atoms.csv
./symmetry_demo_fraction_datasets/sequential_demo_0.0/validation_report.md
./tfm_results_tabpfn/results_random_28entities_demo_0.0/findings.md
./tfm_results_tabpfn/results_random_28entities_demo_0.0/metadata.json
./tfm_results_tabpfn/results_random_28entities_demo_0.0/metrics.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0/predictions.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0_categorical/findings.md
./tfm_results_tabpfn/results_random_28entities_demo_0.0_categorical/metadata.json
./tfm_results_tabpfn/results_random_28entities_demo_0.0_categorical/metrics.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0_categorical/predictions.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0_onehot/findings.md
./tfm_results_tabpfn/results_random_28entities_demo_0.0_onehot/metadata.json
./tfm_results_tabpfn/results_random_28entities_demo_0.0_onehot/metrics.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0_onehot/predictions.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0_shared_embedding/findings.md
./tfm_results_tabpfn/results_random_28entities_demo_0.0_shared_embedding/metadata.json
./tfm_results_tabpfn/results_random_28entities_demo_0.0_shared_embedding/metrics.csv
./tfm_results_tabpfn/results_random_28entities_demo_0.0_shared_embedding/predictions.csv
./tfm_results_tabpfn/results_random_demo_0.0/findings.md
./tfm_results_tabpfn/results_random_demo_0.0/metadata.json
./tfm_results_tabpfn/results_random_demo_0.0/metrics.csv
./tfm_results_tabpfn/results_random_demo_0.0/predictions.csv
./tfm_results_tabpfn/results_random_demo_0.0_onehot/findings.md
./tfm_results_tabpfn/results_random_demo_0.0_onehot/metadata.json
./tfm_results_tabpfn/results_random_demo_0.0_onehot/metrics.csv
./tfm_results_tabpfn/results_random_demo_0.0_onehot/predictions.csv

```

./tfm_results_tabpfn/results_random_demo_0.0_shared_embedding/findings.md
./tfm_results_tabpfn/results_random_demo_0.0_shared_embedding/metadata.json
./tfm_results_tabpfn/results_random_demo_0.0_shared_embedding/metrics.csv
./tfm_results_tabpfn/results_random_demo_0.0_shared_embedding/predictions.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0/findings.md
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0/metadata.json
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0/metrics.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0/predictions.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_categorical/findings.md
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_categorical/metadata.json
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_categorical/metrics.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_categorical/predictions.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_onehot/findings.md
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_onehot/metadata.json
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_onehot/metrics.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_onehot/predictions.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_shared_embedding/findings.md
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_shared_embedding/metadata.json
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_shared_embedding/metrics.csv
./tfm_results_tabpfn/results_sequential_28entities_demo_0.0_shared_embedding/predictions.csv
./tfm_results_tabpfn/results_sequential_demo_0.0/findings.md
./tfm_results_tabpfn/results_sequential_demo_0.0/metadata.json
./tfm_results_tabpfn/results_sequential_demo_0.0/metrics.csv
./tfm_results_tabpfn/results_sequential_demo_0.0/predictions.csv
./tfm_results_tabpfn/results_sequential_demo_0.0_onehot/findings.md
./tfm_results_tabpfn/results_sequential_demo_0.0_onehot/metadata.json
./tfm_results_tabpfn/results_sequential_demo_0.0_onehot/metrics.csv
./tfm_results_tabpfn/results_sequential_demo_0.0_onehot/predictions.csv
./tfm_results_tabpfn/results_sequential_demo_0.0_shared_embedding/findings.md
./tfm_results_tabpfn/results_sequential_demo_0.0_shared_embedding/metadata.json
./tfm_results_tabpfn/results_sequential_demo_0.0_shared_embedding/metrics.csv
./tfm_results_tabpfn/results_sequential_demo_0.0_shared_embedding/predictions.csv

```

Most important raw row-level/result files:

- `tfm_results_tabpfn/*/predictions.csv`
- `relational_pattern_results/*/metrics.csv`
- `relational_pattern_results/*/metrics_by_relation.csv`
- `fb15k237_results/sampled_4k_1k_2k_all_encoders/metrics_by_relation.csv`
- `fb15k237_results/sampled_4k_1k_2k_all_encoders/fb15k237_binary_sample.csv`
- `fb15k237_results/tabpfn_filtered_ranking_path_k3_10/filtered_ranks.csv`
- `fb15k237_results/tabpfn_filtered_ranking_path_k3_10/hourly_status.csv`
- `fb15k237_results/tabpfn_filtered_ranking_categorical_10/filtered_ranks.csv`

No generated datasets or prediction files were deleted.

FB15K-237 TabPFN Improvement Experiments, 2026-05-15

User asked to try the proposed improvements for making TabPFN better at FB15K-237 link prediction:

1. reciprocal queries;
2. hard negatives;
3. mined relation-specific rules instead of hashed path features;
4. type/cardinality/degree features;
5. KGE scores as features;
6. TabPFN as a top-K reranker;
7. more training data.

Created:

- `run_fb15k237_tabpfn_improvement_experiments.py`

What the script implements:

- DistMult KGE baseline/context:
 - reciprocal train triples are used: `(h, r, t)` and `(t, r__inverse, h)`;
 - model is trained with all-tail cross-entropy over the 14,541 FB15K-237 entities;
 - scores are used as:
 - standalone KGE baseline;
 - hard-negative source;
 - numeric TabPFN features;
 - top-K candidate generator.
- Reciprocal TabPFN evaluation:
 - RHS/tail query: `(h, r, ?)`;

- LHS/head query: `(t, r__inverse, ?)`;
 - filtered ranks still use train+valid+test true triples as filters.
- Hard negatives:
 - KGE top candidates;
 - local path candidates;
 - high-degree candidates;
 - relation range candidates;
 - all candidates are filtered against known train triples in reciprocal query form.
- Mined relation-specific rules:
 - length-2 templates: `r(x,y) <- p1(x,z), p2(z,y)`;
 - length-3 templates: `r(x,y) <- p1(x,z1), p2(z1,z2), p3(z2,y)`;
 - mined from train only;
 - ranked by smoothed confidence times log support;
 - stored in `mined_rules.json`.
- Type/cardinality/degree features:
 - entity in/out degree;
 - relation-specific head/tail counts;
 - known tails for `(h,r,?)`;
 - known heads for `(?,r,t)`;
 - domain/range compatibility;
 - relation functionality and inverse functionality.
- KGE feature block:
 - raw KGE score;
 - z-scored KGE score within the query;
 - KGE rank percentile;
 - margin from the top KGE candidate.
- Top-K reranker:
 - KGE generates top 500 filtered-compatible candidates;
 - TabPFN scores only those candidates;
 - final hybrid score is query-normalized KGE score plus `1.0 * logit(TabPFN positive probability)` for top-K candidates;
 - all non-top-K candidates keep their KGE score, so filtered MRR/Hits can still be computed against all entities.

First attempt:

```
python3 run_fb15k237_tabpfn_improvement_experiments.py \
--output-dir fb15k237_results/tabpfn_improvements_20260515_2h \
--eval-triples 5 \
--max-eval-seconds 7200 \
--report-interval-seconds 3600 \
--train-positives 4000 \
--more-train-positives 10000 \
--negatives-per-positive 1 \
--kge-dim 128 \
--kge-epochs 5 \
--kge-batch-size 768 \
--kge-max-train-seconds 900 \
--kge-learning-rate 0.02 \
--rule-mine-positive-samples-per-relation 40 \
--rule-mine-max-paths-per-row 1000 \
--max-rules-per-relation 16 \
--topk 500 \
--rerank-weight 1.0 \
--device cuda \
--n-estimators 2
```

This was stopped manually because the fifth sampled path-hash query became a pathological CPU path-extraction case and blocked progress to the remaining variants. Partial artifacts are preserved:

- `fb15k237_results/tabpfn_improvements_20260515_2h/filtered_ranks.csv`
- completed rows:
 - `00_kge_only`: 10 rank rows, meaning 5 triples / 10 RHS+LHS queries;
 - `01_reciprocal_path_hash_k3`: 8 rank rows, meaning 4 triples / 8 RHS+LHS queries.

Final budgeted run:

```
python3 run_fb15k237_tabpfn_improvement_experiments.py \
--output-dir fb15k237_results/tabpfn_improvements_20260515_2h_eval4 \
--eval-triples 4 \
--max-eval-seconds 7200 \
--report-interval-seconds 3600 \
--train-positives 4000 \
--more-train-positives 10000 \
--negatives-per-positive 1 \
--kge-dim 128 \
--kge-epochs 5 \
--kge-batch-size 768 \
--kge-max-train-seconds 900 \
--kge-learning-rate 0.02 \
--rule-mine-positive-samples-per-relation 40 \
--rule-mine-max-paths-per-row 1000 \
--max-rules-per-relation 16 \
--topk 500 \
--rerank-weight 1.0 \
--device cuda \
--n-estimators 2
```

Run duration:

- final elapsed evaluation time: 6463.470377 seconds = 1.80 hours;
- all variants completed the same 4 fixed test triples = 8 filtered queries.

KGE training:

- reciprocal train rows: 544230;
- entities: 14541;
- reciprocal relations: 474;
- epochs: 5;
- printed losses:
 - epoch 1: 4.6304;
 - epoch 2: 4.1155;
 - epoch 3: 3.9402;
 - epoch 4: 3.8847;
 - epoch 5: 3.6370.

Final aggregate filtered ranking metrics on the 4-triple / 8-query sample:

variant	proposal tested	eval queries	filtered MRR	H@1	H@3	H@10	RHS MRR	RHS H@10	LHS MRR	LHS H@10
00_kge_only	KGE baseline/context	8	0.450198	0.375	0.500	0.625	0.400000	0.750	0.500395	0.500395
01_reciprocal_path_hash_k3	reciprocal queries + existing path-hash encoder	8	0.140035	0.125	0.125	0.125	0.275845	0.250	0.004225	0.004225
02_hardneg_path_hash_k3	hard negatives + reciprocal path-hash	8	0.002591	0.000	0.000	0.000	0.003938	0.000	0.001244	0.001244
03_mined_rules	mined relation-specific path rules	8	0.357255	0.250	0.500	0.500	0.380871	0.500	0.333638	0.500395
04_rules_type_degree	mined rules + type/cardinality/degree features	8	0.032320	0.000	0.000	0.125	0.014402	0.000	0.050238	0.250395
05_rules_type_degree_kge_features	add KGE scores as TabPFN features	8	0.001049	0.000	0.000	0.000	0.002005	0.000	0.000094	0.000395
06_topk_tabpfn_reranker	KGE top-500 + TabPFN reranking	8	0.264460	0.250	0.250	0.250	0.271377	0.250	0.257544	0.250395
07_more_training_data_topk	top-K reranking with 10k train positives	8	0.261683	0.250	0.250	0.250	0.266720	0.250	0.256647	0.250395

Feature/training metadata:

variant	supervised rows	features	feature time s	fit time s
00_kge_only	n/a	0	0.00	0.00
01_reciprocal_path_hash_k3	16000	995	146.30	10.68
02_hardneg_path_hash_k3	16000	995	183.61	7.58

variant	supervised rows	features	feature time s	fit time s
03_mined_rules	16000	538	7.65	7.73
04_rules_type_degree	16000	550	8.91	6.10
05_rules_type_degree_kge_features	16000	554	20.70	4.12
06_topk_tabpfn_reranker	16000	554	20.38	6.12
07_more_training_data_topk	40000	554	56.54	14.08

Per-query rank rows are saved in:

- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/filtered_ranks.csv

Top-K diagnostic for 06_topk_tabpfn_reranker:

- target was inside top 500 for 6 of 8 queries = 75%;
- target was outside top 500 for:
 - query 1 LHS, KGE rank 2966;
 - query 3 LHS, KGE rank 804.

Top-K diagnostic for 07_more_training_data_topk:

- target was inside top 500 for 6 of 8 queries = 75%;
- same two LHS queries were outside top 500.

Important interpretation:

- These are very small-sample filtered-ranking results, not final FB15K-237 numbers. They are useful for debugging design directions but should not be compared as stable benchmark estimates against the full [facebookresearch/kbc](#) numbers.
- The strongest result in this run is the DistMult KGE baseline itself: MRR **0.450198**, H@10 **0.625** on this 8-query sample.
- Among pure TabPFN-style encoders tested here, mined relation-specific path rules are the most promising: MRR **0.357255**, H@10 **0.500**.
- Reciprocal path-hash improves the previous conceptual setup but remains much weaker and very slow: MRR **0.140035**, H@10 **0.125**.
- Hard negatives with path-hash features hurt badly in this configuration: MRR **0.002591**. The likely issue is not that hard negatives are intrinsically bad, but that the path-hash TabPFN probability scale and training distribution become poorly calibrated for all-entity ranking.
- Adding type/degree features naively also hurt here. Those features may need normalization by relation and/or a rank-aware training objective; raw count/log-count features can dominate TabPFN in unhelpful ways.
- Adding KGE scores as ordinary TabPFN features failed in all-candidate scoring: MRR **0.001049**. This indicates that simply giving TabPFN a KGE score column does not guarantee that it preserves the KGE ordering.
- The top-K reranker underperformed KGE-only: MRR **0.264460** vs **0.450198**. Where KGE already ranked targets highly, the $1.0 * \text{logit}(\text{TabPFN})$ rerank term often moved the target down. This means the reranker needs calibration/tuning, for example:
 - much smaller rerank weight;
 - validation-tuned blending;
 - pairwise/listwise ranking loss outside TabPFN;
 - train rows shaped as top-K candidate lists, not standalone binary rows.
- More training positives did not help the top-K reranker in this setup: MRR **0.261683**, essentially the same as **06_topk_tabpfn_reranker**.
- The practical conclusion is that KGE/path-rule candidate generation is necessary, but TabPFN does not automatically become a good ranker just because the candidate features are more relational. The next principled experiment should tune the hybrid score on validation queries and train TabPFN only on candidates drawn from the same top-K distribution used at evaluation.

New artifacts:

- run_fb15k237_tabpfn_improvement_experiments.py
- fb15k237_results/tabpfn_improvements_20260515_2h/
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/eval_triples.csv
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/filtered_ranking_metrics.csv
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/filtered_ranks.csv
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/hourly_status.csv
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/metadata.json
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/mined_rules.json
- fb15k237_results/tabpfn_improvements_20260515_2h_eval4/*_supervised_rows.csv

FB15K-237 ComplEx-As-Encoder Experiments, 2026-05-16

User asked to try all three ComplEx-based ideas:

- ComplEx-only;
- frozen ComplEx embeddings/interactions as a TabPFN encoder, with TabPFN-only scores;
- ComplEx backbone plus TabPFN residual reranking.

Created:

- run_fb15k237_complex_tabpfn_experiments.py

Implementation details:

- ComplEx was trained/evaluated using reciprocal relations:
 - RHS query: $(h, r, ?)$;
 - LHS query: $(t, r_inverse, ?)$.
- Filtered ranking uses train+valid+test true triples, matching the KBC protocol.
- ComplEx model:
 - rank: 500;
 - train rows: all 544230 reciprocal FB15K-237 train rows;
 - optimizer: Adagrad;
 - learning rate: 0.1;
 - N3-style regularization weight: 0.05;
 - epochs: 30;
 - batch size: 512;
 - checkpoint: fb15k237_results/complex_tabpfn_20260515_2h/complex_checkpoint.pt.
- TabPFN ComplEx encoder:
 - frozen rank-500 ComplEx model remains the source model;
 - TabPFN receives only the first 16 latent dimensions plus aggregate ComplEx interaction features;
 - final TabPFN feature count: 90;
 - features include query real/imag components, candidate tail real/imag components, per-dimension score terms, total ComplEx score, norms, cosine, and simple dot/difference aggregates.
- Residual score:

```
final_score = complex_score + alpha * logit(tabpfn_probability)
```

- **alpha** was tuned on 80 validation triples = 160 RHS+LHS validation queries.

Large TabPFN-sample attempts:

- 200000 TabPFN rows, feature dim 64, 2 estimators:
 - output dir: fb15k237_results/complex_tabpfn_20260515_2h;
 - stopped after about an hour because it had not reached validation/test evaluation.
- 100000 TabPFN rows, feature dim 64, 1 estimator:
 - output dir: fb15k237_results/complex_tabpfn_20260515_2h_100k;
 - stopped because it had not reached evaluation quickly enough.
- 20000 TabPFN rows, feature dim 16, 1 estimator:
 - output dir: fb15k237_results/complex_tabpfn_20260515_2h_20k_d16;
 - stopped because validation/inference was still too slow.
- Final completed run used 5000 TabPFN rows. This was the largest setting that completed the requested two-hour evaluations in this environment.

Final completed command:

```
python3 run_fb15k237_complex_tabpfn_experiments.py \
  --output-dir fb15k237_results/complex_tabpfn_20260515_2h_5k_d16 \
  --complex-checkpoint fb15k237_results/complex_tabpfn_20260515_2h/complex_checkpoint.pt \
  --variants all \
  --max-seconds-per-variant 7200 \
  --report-interval-seconds 3600 \
  --progress-every-triples 1000 \
  --complex-rank 500 \
  --complex-epochs 30 \
  --complex-batch-size 512 \
  --complex-learning-rate 0.1 \
  --complex-reg 0.05 \
  --complex-init 0.001 \
  --tabpfn-train-original-positives 5000 \
  --tabpfn-max-train-rows 5000 \
  --tabpfn-negatives-per-positive 1 \
  --tabpfn-feature-dim 16 \
  --tabpfn-topk 500 \
  --alpha-tune-triples 80 \
  --alpha-grid 0,0.001,0.003,0.01,0.03,0.1,0.3,1.0 \
  --device cuda \
  --n-estimators 1
```

Alpha tuning result:

alpha	validation queries	validation MRR	H@1	H@3	H@10
0.100	160	0.323410	0.25000	0.33750	0.48125
0.030	160	0.322077	0.24375	0.34375	0.48750
0.003	160	0.321392	0.24375	0.34375	0.49375

alpha	validation queries	validation MRR	H@1	H@3	H@10
0.010	160	0.321283	0.24375	0.34375	0.49375
0.001	160	0.320891	0.24375	0.33750	0.49375
0.000	160	0.317763	0.23750	0.33750	0.49375
0.300	160	0.299087	0.23125	0.30625	0.43125
1.000	160	0.242704	0.19375	0.23750	0.32500

Selected alpha:

alpha = 0.1

Final filtered ranking results:

variant	evaluation budget/result	triples	RHS+LHS queries	filtered MRR	H@1	H@3	H@10
complex_only	completed full test in 0.53h	20466	40932	0.357899	0.263022	0.393286	0.549350
complex_tabpfn_only	2h cap	1044	2088	0.086184	0.074234	0.086686	0.106801
complex_tabpfn_residual	2h cap	3348	6696	0.356750	0.262694	0.393369	0.549134

Side-specific final results:

variant	RHS MRR	RHS H@10	LHS MRR	LHS H@10
complex_only	0.452492	0.647953	0.263307	0.450748
complex_tabpfn_only	0.130443	0.153257	0.041925	0.060345
complex_tabpfn_residual	0.451848	0.655615	0.261651	0.442652

Same-prefix comparisons:

- On the first 1044 triples, where TabPFN-only was evaluated:
 - ComplEx-only MRR: 0.377030;
 - ComplEx-only H@10: 0.556034;
 - TabPFN-only MRR: 0.086184;
 - TabPFN-only H@10: 0.106801.
- On the first 3348 triples, where residual reranking was evaluated:
 - ComplEx-only MRR: 0.362548;
 - ComplEx-only H@10: 0.553017;
 - residual MRR: 0.356750;
 - residual H@10: 0.549134.

Residual diagnostic:

- top-500 ComplEx candidate set contained the target for 92.951% of residual-evaluated RHS/LHS queries;
- alpha=0.1 was used for all residual rows.

Interpretation:

- ComplEx-only basically reproduces the KBC-codebase scale:
 - KBC README rank-500 FB15K-237 reference is approximately MRR 0.36, H@10 0.56;
 - our ComplEx-only run gives MRR 0.357899, H@10 0.549350.
- TabPFN-only on ComplEx features is not competitive:
 - MRR 0.086184, H@10 0.106801 after 2h;
 - it destroys the ComplEx ranking even though the features contain ComplEx information.
- ComplEx + TabPFN residual reranking is close to ComplEx, but does not improve it on the test prefix:
 - residual MRR 0.356750, H@10 0.549134;
 - same-prefix ComplEx-only MRR 0.362548, H@10 0.553017.
- Validation selected a nonzero alpha (0.1), but this did not transfer to a test-prefix improvement.
- The practical conclusion is that ComplEx is already the right inductive bias for FB15K-237. TabPFN can be made non-destructive only when used as a small residual, but with the current binary/top-K training setup it does not add useful ranking information beyond ComplEx.

Artifacts:

- run_fb15k237_complex_tabpfn_experiments.py
- fb15k237_results/complex_tabpfn_20260515_2h/complex_checkpoint.pt
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/filtered_ranking_metrics.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/filtered_ranks.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/complex_only_filtered_ranking_metrics.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/complex_only_filtered_ranks.csv

- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/complex_tabpfn_only_filtered_ranking_metrics.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/complex_tabpfn_only_filtered_ranks.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/complex_tabpfn_residual_filtered_ranking_metrics.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/complex_tabpfn_residual_filtered_ranks.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/alpha_tuning_metrics.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/alpha_tuning_ranks.csv
- fb15k237_results/complex_tabpfn_20260515_2h_5k_d16/metadata.json